

Universidad Autónoma de Baja California.

Facultad de Ciencias Químicas e Ingeniería.



M5.2_Diseño de proyecto

Alumno:

Cordova Contreras Cesar. #1284551

Diseño de la Aplicación, Proyecto Integrador ElotApp

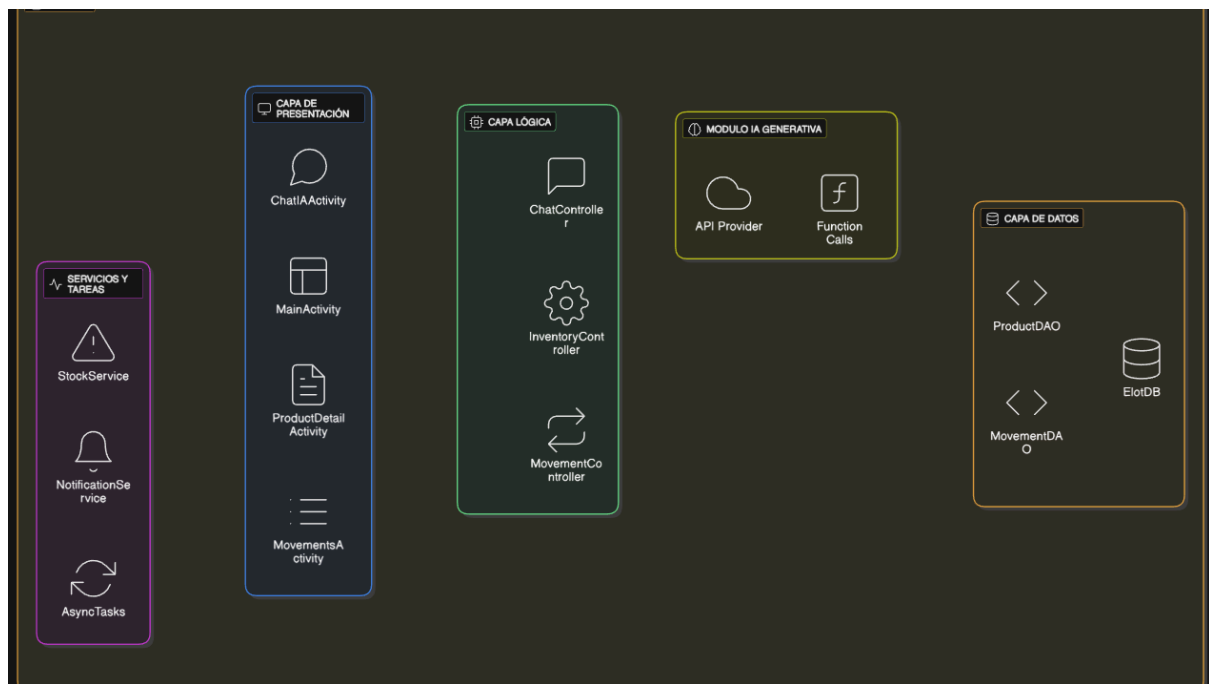
1. Introducción

El presente documento describe el diseño técnico y funcional de la aplicación móvil ElotApp, la cual permite a una vendedora de elotes controlar su inventario de manera sencilla y realizar consultas inteligentes mediante IA generativa.

Este documento es una extensión del proyecto integrador propuesto, y busca representar visualmente la estructura del sistema y explicar la función de cada componente.

2. Arquitectura General

Diagrama General del Sistema



3. Descripción de Componentes

Capa de Presentación (UI)

1. MainActivity:

Pantalla principal con la lista de productos. Usa ListView y un ArrayAdapter para mostrar nombre, cantidad y color de estado (verde, amarillo, rojo según nivel de stock).

2. ProductDetailActivity:

Permite agregar o editar productos. Contiene campos de texto y botones de guardar o eliminar.

3. MovementsActivity:

Registra las entradas, salidas o ventas, con fecha y cantidad.

4. ChatIAActivity:

Interfaz de conversación con la IA generativa. El usuario escribe una pregunta y recibe una respuesta basada en los datos del inventario.

Capa Lógica

- **InventoryController:** Gestiona el CRUD de productos (crear, leer, actualizar, eliminar).
- **MovementController:** Registra y consulta los movimientos.
- **ChatController:** Recibe la pregunta del usuario, la envía a la API de IA Gemini interpreta la intención y consulta los datos en SQLite para generar la respuesta.

Capa de Datos

- **SQLiteDatabase (ElotDB):**
Almacena toda la información localmente.
 - products → datos de cada producto.
 - movements → historial de entradas/salidas/ventas.
- **DAOs:**
Métodos específicos de consulta, como:
 - getStock(productName)
 - getSalesByPeriod(period)
 - getLowStockItems()

Servicios y Tareas Asíncronas

- **StockService:** Verifica periódicamente el inventario y detecta productos con bajo stock.
- **NotificationService:** Lanza notificaciones con Notification.Builder.
- **AsyncTasks:** Se usan para cálculos o consultas largas (por ejemplo, el total de ventas de la semana) sin bloquear la interfaz del usuario.

Módulo de IA Generativa

Permite realizar preguntas en lenguaje natural sobre el inventario o las ventas.

Flujo:

1. Usuario: “¿Cuántos elotes tengo?”
2. El modelo Gemini analiza la intención (get_stock("elotes")).
3. Se ejecuta una función local que consulta SQLite.
4. Se envía la respuesta al modelo.
5. El modelo responde de forma natural:
ejemplo: Tienes 126 elotes en inventario.

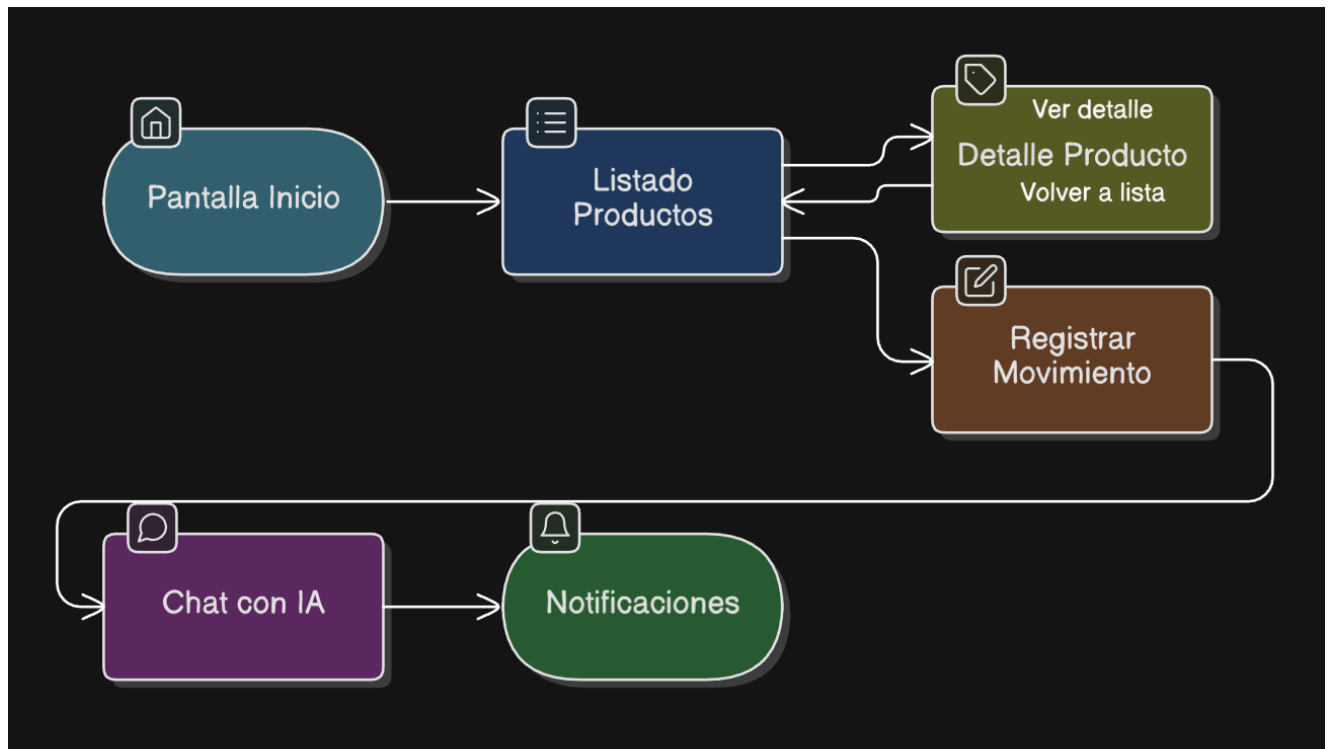
Funciones principales:

- get_stock(product)
- get_sales(period, product?)
- get_low_stock()

Proveedores posibles:

- Google Gemini API (con Function Calling).
- OpenAI GPT API (con Tool Calling).

4. Flujo de Navegación



5. Integración con los Módulos del Curso

Módulo	Aplicación en ElotApp
M1.1 – Introducción a Android	Configuración del entorno y creación del proyecto.
M1.2 – Ciclo de vida	Manejo de estados y restauración de datos.
M2 – Múltiples tareas	Navegación entre Activities y paso de datos.
M3 – Interfaz de usuario	Diseño adaptable con ConstraintLayout y Fragmentos.
M4.1 – Listas y adaptadores	Uso de ListView para mostrar productos.
M4.2 – SQLite y cursores	Almacenamiento local de datos.
M4.3 – Tareas asíncronas	Consultas en segundo plano.
M4.4 – Fragmentos	Interfaz adaptable para móvil/tableta.
M5.1 – Servicios	Notificaciones de bajo stock.

6. Conclusión

El diseño de ElotApp se fundamenta en los principios del desarrollo Android vistos en el curso, integrando todos los módulos en una aplicación práctica y funcional.

La incorporación de IA generativa añade un valor adicional, permitiendo que el usuario consulte información del inventario con lenguaje natural de forma rápida y accesible.